03.2 Otros métodos de agrupación - Clustering Jerárquico y Densidad

Print to PDF

```
import matplotlib.pyplot as plt
%matplotlib inline
import pandas as pd
import numpy as np
from skimage import io
from IPython import display
```

En este cuaderno vamos a necesitar dibujar un dendograma

Disponible en el "**03.0ClusteringUtilidades.ipynb**"

```
run 03.0_ClusteringUtilidades.ipynb
```

```
load done!
```

Descripción general del método de agrupación jerárquica

- A diferencia de **K-means**, son métodos **deterministas**.
- Parten de una matriz $D = (d_{ij})$ de **distancias** o $S = (s_{ij})$ de **similitudes**, matrices $N \times N$ entre los N elementos del conjunto de instancias.
- Si todas las variables son **continuas**, la distancia más utilizada es la euclídea (con variables estandarizadas univariantemente en caso de ser conveniente).
- Si hay variables **continuas** y **categóricas** no suele ser aceptable trabajar con distancias y se usan **similitudes**.

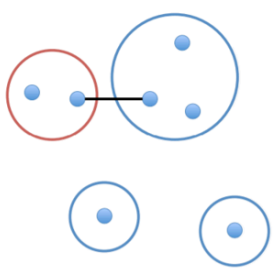
El algoritmo tiene los siguientes pasos:

- Comenzamos con **N grupos** (uno para cada instancia) y la matriz *D* o *S*.
- Buscamos los elementos más **próximos** (mínimo valor d_{ij} o s_{ij}) y los unimos en un grupo.
- Recalculamos la matriz de distancias (o similitudes) definiendo una **distancia o similitud entre este nuevo grupo** y el resto.
- Repetimos los dos pasos anteriores hasta que todas las observaciones estén unidas en un solo grupo

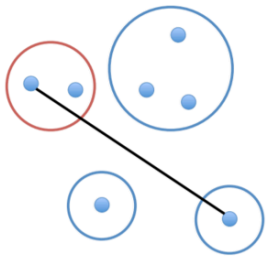
La forma de calcular la distancia entre grupos de puntos puede hacerse de diferentes formas como:

- Por **encadenamiento simple** tomando como distancia de 2 grupos la de los 2 puntos más próximos. Tiende a **encadenar individuos sueltos** y no grupos.
- Por **encadenamiento completo** tomando como distancia de 2 grupos la de los 2 puntos más alejados. Es menos sensible a **atípicos**.
- Método de **Ward** que **maximiza la homogeneidad** intra-grupo. Como medida de homogeneidad se usa la **suma de cuadrados** de los errores. Tiende a formar grupos esféricos aunque los grupos no lo sean.

Una forma de visualizar la estructura del grupo gerárquico es con el **dendograma**.



Encadenamiento simple



Encadenamiento completo

Ejemplo simple de una agrupación jerárquica

Un ejemplo simple del **proceso de agrupación jerárquica** donde se utiliza la **distancia euclídea** para calcular la matriz de distancias se muestra a continuación.

En cada paso se forma un nuevo grupo que implica el **recálculo de las coordenadas de su centroide (en verde)**.

Agrupación	X	Y
A	1	1
B	2	1
C	4	5
D	7	7
E	5	7

Distancias	A	B	C	D	E
A	0				
B	**1**	0			
C	5	4,5	0		
D	8,5	7,8	3,6	0	
E	7,2	6,7	2,2	2	0

Agrupación	X	Y
AB	1,5	1
C	4	5
D	7	7
E	5	7

Distancias	AB	C	D	E
AB	0			
C	4,7	0		
D	8,1	3,6	0	
E	6,9	2,2	**2**	0

Agrupación	X	Y
AB	1,5	1
C	4	5
DE	6	7

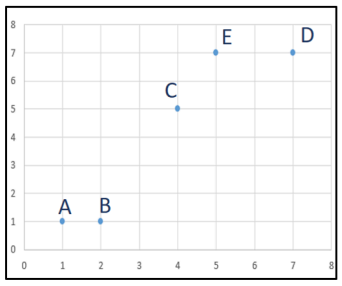
Distancias	AB	C	DE
AB	0		
C	4,7	0	
DE	7,5	**2,8**	0

Agrupación	X	Y
AB	1,5	1
CDE	5,33	6,33

Distancias	AB	C
AB	0	
C	**6,4**	0

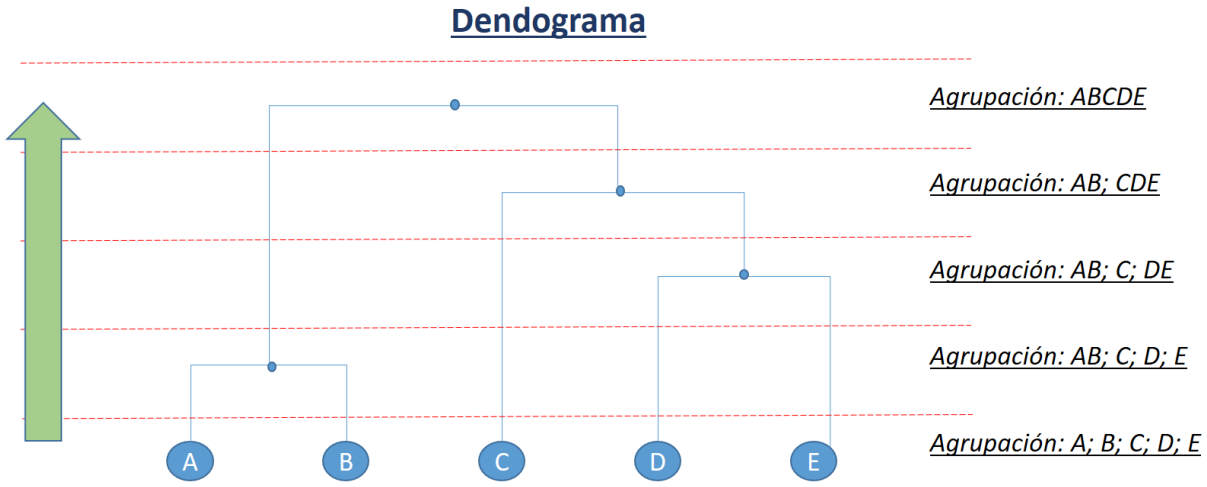
Agrupación	X	Y
ABCDE	3,80	4,20

Distancias	ABCDE
ABCDE	0



Dendograma con el resultado del ejemplo

Se muestran las posibles soluciones en función del nivel de agrupación



Caso de uso de la similitud

En el caso de usar una similitud el algoritmo sería igual pero sustituyendo la matriz de distancia por una de similitud. La diagonal principal serían *unos* y el cálculo de las similitudes entre dos elementos sería como se indica en el ejemplo que aparece a continuación.

Cluster jerárquico sobre el conjunto Iris

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target']=iris['target']
df.head()
X = df.values[:,0:4]
y = df.values[:,4]
```

Se resuelve un cluster aglomerativo con **scikit-learn** que es **AgglomerativeClustering**, cuyos parámetros más importantes son:

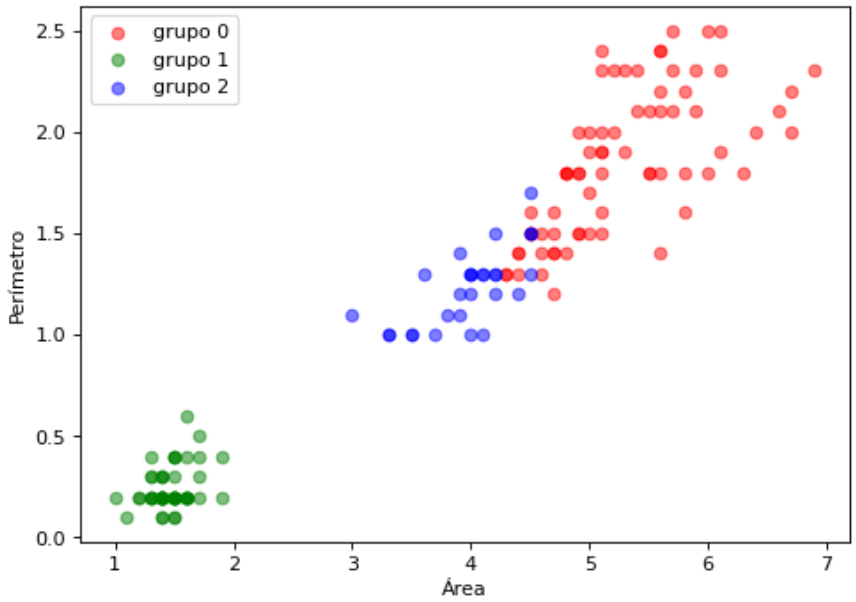
- **n_clusters** (int o None, default=2). El número de clústeres a buscar. Debe ser None si distance_threshold no es None.
- **affinity** (str o función llamable, default='euclidean'). Métrica utilizada. Puede ser "euclidian", "l1", "l2", "manhattan", "cosine" o "precomputed". Si la linkage es "ward", solo se acepta "euclidean". Si es "precomputed", se necesita una matriz de distancia (en lugar de una matriz de similitud) como entrada para el método de ajuste.
- **compute_full_tree** ('auto' or bool, default='auto'). Para la ejecución hasta construir n_clusters.
- **linkage** ('ward', 'complete', 'average', 'single', default='ward').
- **distance_threshold** (float, default=None), es el umbral de distancia de vinculación por encima del cual los clústeres no se fusionarán. Si no es None, n_clusters debe ser None y compute_full_tree debe ser True.
- **compute_distances** (bool, default=False). Calcula las distancias entre los clústeres incluso si no se utiliza la distancia umbral (distance_threshold). Sirve para hacer la visualización de dendrogramas, pero introduce una sobrecarga computacional y de memoria. (Nuevo en versión 0.24)

Según los valores de linkage:

- 'ward' **minimiza la varianza** de los grupos que se fusionan.
- 'average' usa el **promedio de las distancias** de cada observación de los dos conjuntos.
- 'complete' utiliza **las distancias máximas** entre todas las observaciones de los dos conjuntos.
- 'single' utiliza el **mínimo de las distancias** entre todas las observaciones de los dos conjuntos.

```
from sklearn.cluster import AgglomerativeClustering
ac = AgglomerativeClustering(n_clusters=3, affinity='euclidean',
linkage='complete')
y_ag = ac.fit_predict(X)
etiquetas=np.unique(y_ag)
```

```
import matplotlib.pyplot as plt
names = ['grupo 0', 'grupo 1', 'grupo 2']
marcas = ['*', 'o', 's']
color = ['red', 'green', 'blue']
plt.figure(figsize=(7, 5), dpi=80)
for i in range(len(names)):
    cl = i
    plt.scatter(X[y_ag==cl,2], X[y_ag==cl,3], c=color[i], alpha=0.5,
marker='o', label=names[i])
plt.xlabel("Área")
plt.ylabel("Perímetro")
plt.legend(loc='upper left')
plt.show()
```



Es posible visualizar el dendograma con la siguiente ejecución

- Para que se pueda dibujar el dendograma es necesario que **AgglomerativeClustering** construya la variable **distances**_. Hay que dar una combinación de parámetros que lo permita.

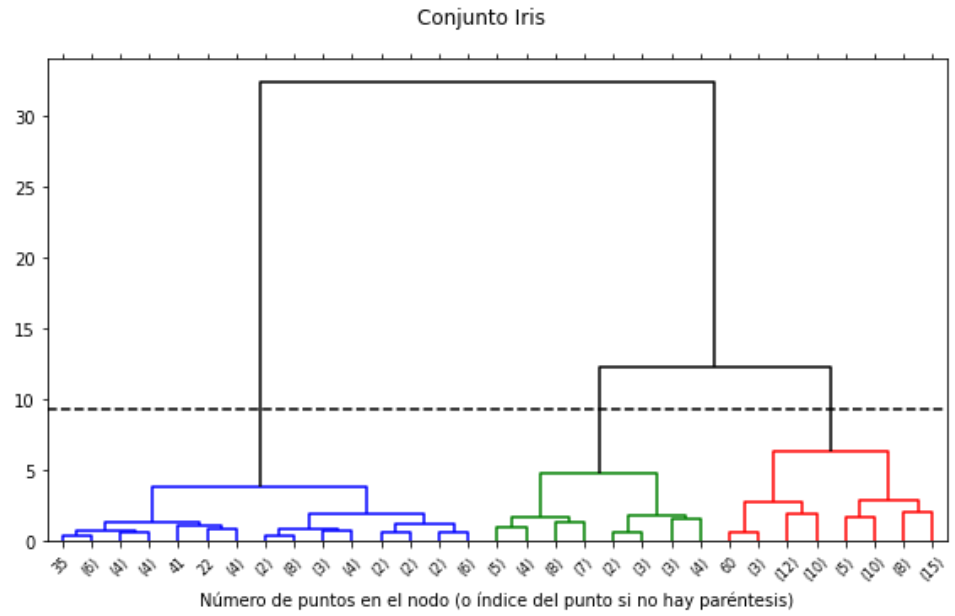
Las variables que maneja el cluster jerarquico y que intervienen en la construcción del dendograma tienen la siguiente estructura:

- **distances**_ : Es una lista con las distancias a las que se han ido efectuando las uniones por orden de aparición en el dendograma. Ocurren de menor distancia a mayor distancia. En el dendograma la escala de las distancias aparece en el eje vertical.
- **children**_ : Es una lista con los pares de elementos que se funden en cada nodo del dendograma. Por cada item de distancia aparece un item con el nodo. El número de filas del conjunto X se corresponde con el nº de elementos de devuelve fit_predict y que se vuelcan en **etiquetas**, supongamos **N**. Cuando en el nodo se unen dos elementos individuales en **children**_ aparecen dos números entre **0 y N-1**, pero si unen nodos de agrupaciones anteriores a aperecen valores **N o mayor que N**.

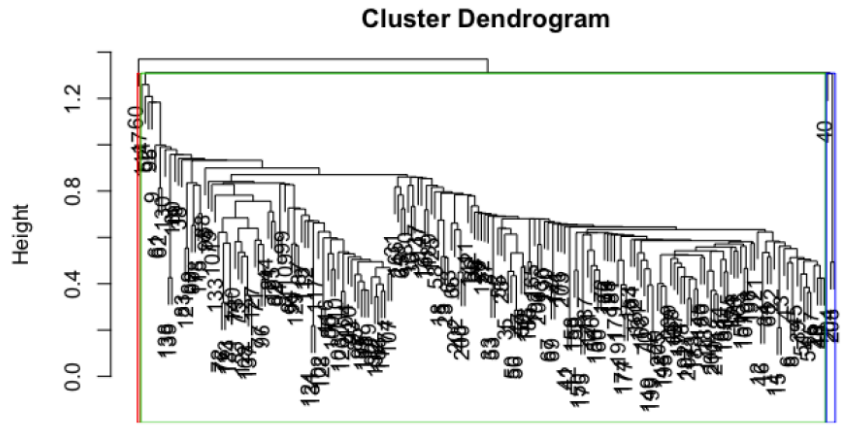
```
cluster_dist = AgglomerativeClustering(distance_threshold=None,
n_clusters=3, compute_distances=True)
cluster_dist.fit(X)
```

```
AgglomerativeClustering(compute_distances=True, n_clusters=3)
```

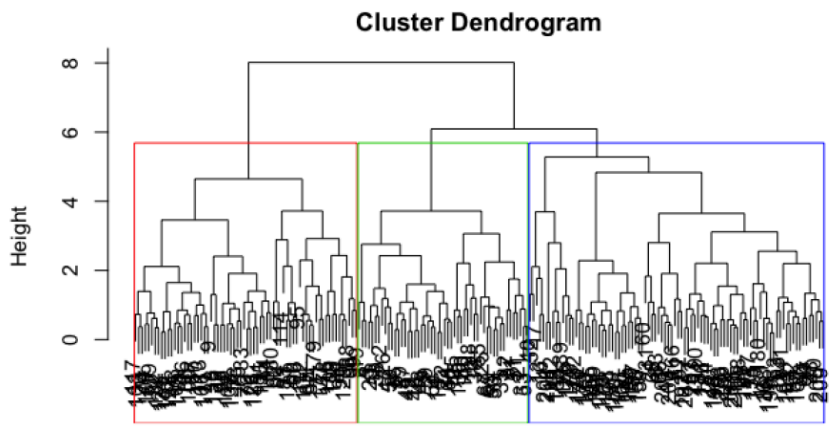
```
titulo="Conjunto Iris"
subtitulo="Número de puntos en el nodo (o índice del punto si no hay
paréntesis)"
figSize=(8,5)
plot_dendrogram(cluster_dist, titulo, subtitulo, figSize,
truncate_mode='level', p=4)
```



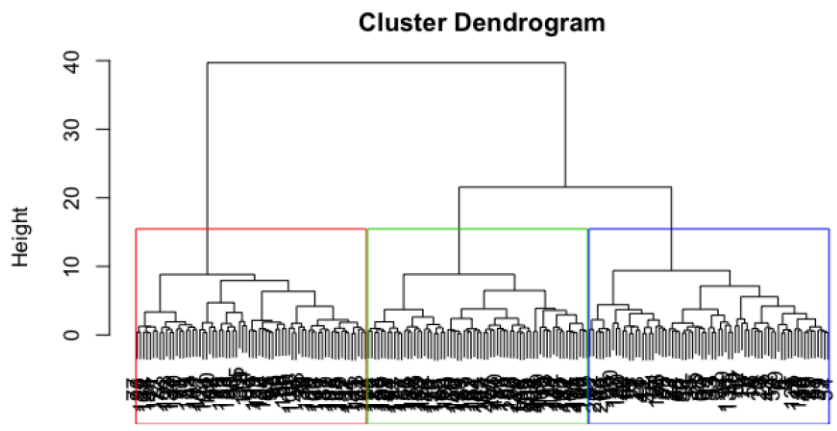
El encadenamiento simple: tiende a **encadenar individuos sueltos** y no grupos.



El encadenamiento completo: es menos sensible a **atípicos**.

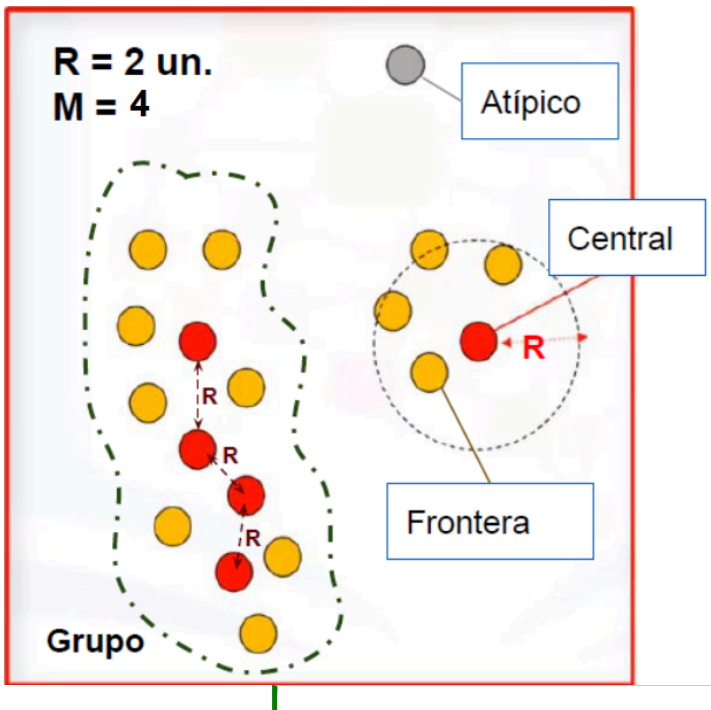


Ward tiende a **formar grupos esféricos** aunque los grupos no lo sean



Clustering por densidad. DBSCAN

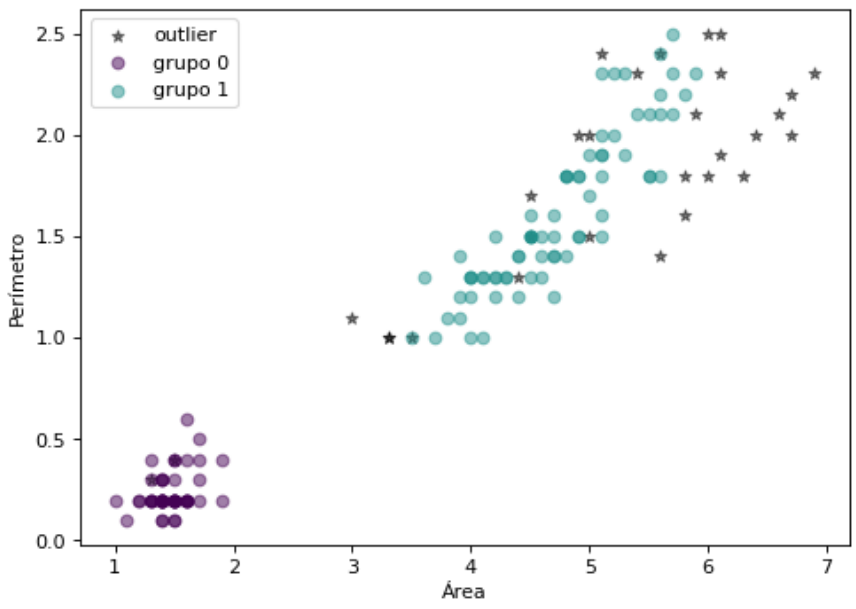
Un cuarto modo de clustering, también determinístico, es el **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)**. Agrupa áreas de alta densidad quitando los puntos atípicos o outliers. Cada grupo se forma por la unión de puntos centrales R-vecinos junto con sus puntos frontera. Los puntos outliers se descartan. La densidad se basa en R, el radio de vecindad y M, mínimo número de vecinos para definir un cluster. Un punto se denomina central si dentro de su R-vecindad hay al menos M puntos. Un punto se denomina frontera si está en la R-vecindad de un central, pero no tiene M R-vecinos. Un punto atípico o outlier es aquel que ni es central, ni frontera.




```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=10, metric='euclidean')
#db = DBSCAN(eps=1, min_samples=40, metric='euclidean')
y_db = db.fit_predict(X)
np.unique(y_db)
```

```
array([-1,  0,  1])
```

```
import matplotlib.pyplot as plt
from matplotlib import cm
viridis = cm.get_cmap('viridis', len(np.unique(y_db)))
plt.figure(figsize=(7, 5), dpi=80)
for i in np.unique(y_db):
    cl = i
    etiqueta="grupo " + str(i)
    if cl==-1:
        plt.scatter(X[y_db==cl,2], X[y_db==cl,3], c="k", alpha=0.5,
marker='*', label="outlier")
    else:
        plt.scatter(X[y_db==cl,2], X[y_db==cl,3], color=viridis(i),
alpha=0.5, marker='o', label=etiqueta)
plt.xlabel("Área")
plt.ylabel("Perímetro")
plt.legend(loc='upper left')
plt.show()
```



Previous
03.1 Aprendizaje no supervisado. Clustering

Next
03.3 Clustering. Análisis de microarrays

By Departamento de Matemática Aplicada
© Copyright 2021.